Compiler Design and Implementation

CS306 — Compilers Laboratory

Assignment 5

---

1. Introduction

A compiler is a system program that translates programs written in a high-level programming language into a lower-level representation that can be executed by a target machine. A complete compiler must perform several stages of processing, including lexical analysis, syntax analysis, semantic analysis, intermediate-code generation, and target-code generation.

This project implements a compiler for the language used in the CS306 Compilers Laboratory. The compiler accepts source programs containing variable declarations, expressions, conditional statements, loops, functions, function calls, input/output operations, and return statements. The implementation translates these programs through multiple intermediate representations before producing MIPS/SPIM assembly code.

The implementation is organized as a pipeline:

Source Program → Lexical Analysis → Parsing → Symbol Table / Semantic Analysis → AST → Three-Address Code → RTL → MIPS Assembly

The project uses Flex-style lexical specifications and a Yacc/Bison-style grammar for the front end. The back end is implemented in C++ using a hierarchy of classes representing AST nodes, TAC statements, RTL statements, and assembly statements. The final compiler driver in `main.cc` coordinates all stages and can additionally produce token, AST, and RTL outputs.

---

2. Objectives

The primary objectives of the project are:

1. To implement a lexical analyzer for the source language.
2. To implement a parser capable of recognizing the language grammar.
3. To construct an Abstract Syntax Tree (AST) representing the parsed program.
4. To maintain symbol tables for variables, functions, parameters, and scopes.
5. To perform semantic validation and type checking.
6. To generate Three-Address Code (TAC) from the AST.
7. To translate TAC into a lower-level Register Transfer Language (RTL).
8. To perform simple register allocation during RTL generation.
9. To generate MIPS/SPIM assembly code from the RTL.
10. To support functions, function calls, stack management, input/output, control flow, integer operations, floating-point operations, strings, and boolean expressions.

The final compiler is therefore not limited to syntax recognition; it demonstrates the complete structure of a small optimizing-oriented compiler backend.

---

3. Overall Compiler Architecture

The compiler is divided into the following major components:

- **Lexical Analyzer:** `a5.l`
- **Parser:** `a5.y`
- **Symbol Table:** `symbol_table.hh`, `symbol_table.cc`
- **Abstract Syntax Tree:** `ast.hh`, `ast.cc`
- **Three-Address Code:** `tac.hh`, `tac.cc`
- **Register Transfer Language:** `rtl.hh`, `rtl.cc`
- **Assembly Representation:** `asm.hh`, `asm.cc`
- **Compiler Driver:** `main.cc`

The Assignment 5 directory contains these source files together with generated lexer/parser files and valid test cases.

The central compilation pipeline is implemented explicitly in `main.cc`. After parsing and AST validation, the compiler performs:

1. AST generation
2. TAC generation
3. RTL generation
4. Assembly generation

This can be summarized as:

```
  Source Program
        |
        v
+----------------+
| Lexical        |
| Analysis       |
|     a5.l        |
+----------------+
        |
        v
+----------------+
| Syntax         |
| Analysis       |
|     a5.y        |
+----------------+
        |
        v
+----------------+
| Symbol Table   |
```

```
| Semantic Check |
+----------------+
        |
        v
+----------------+
|     AST        |
|   ast.cc/hh    |
+----------------+
        |
        v
+----------------+
|      TAC       |
|   tac.cc/hh    |
+----------------+
        |
        v
+----------------+
|      RTL       |
|   rtl.cc/hh    |
+----------------+
        |
        v
+----------------+
| MIPS Assembly  |
|   asm.cc/hh    |
+----------------+
        |
        v
   .spim File
```

The compiler driver constructs `TAC_Code`, `RTL_Code`, and `Asm_Code` objects and invokes the corresponding generation functions sequentially.

---

4. Lexical Analysis

4.1 Lexical Analyzer

The lexical analyzer is implemented in `a5.l`. It uses Flex-style regular expressions to convert the input character stream into tokens consumed by the parser.

The lexer maintains line information using `%option yylineno` and disables the requirement for a user-defined `yywrap()` using `%option noyywrap`.

The lexical specification defines reusable patterns:

- `digit` — decimal digits
- `letter` — alphabetic characters and underscore
- `whitespace` — spaces, tabs, carriage returns, and newlines

Comments beginning with `//` are ignored.

4.2 Keywords

The lexer recognizes the following keywords:

- `int`
- `float`
- `void`
- `string`
- `bool`
- `read`
- `print`
- `if`
- `else`
- `do`
- `while`
- `return`

These are converted into dedicated parser tokens. For example, `int` produces the `INTEGER` token, while `while` produces the `WHILE` token.

4.3 Operators and Punctuation

The lexer recognizes arithmetic operators:

- `+`
- `-`
- `*`
- `/`

Relational operators:

- `<`
- `<=`
- `>`
- `>=`
- `==`
- `!=`

Logical operators:

- `&&`
- `||`
- `!`

It also recognizes assignment, delimiters, and ternary-expression symbols such as:

- `=`
- `;`

- `,`
- `(`
- `)`
- `{`
- `}`
- `?`
- `:`

4.4 Literals and Identifiers

The lexer supports:

- Integer constants
- Floating-point constants
- String constants
- Identifiers

Integer and floating-point constants are converted into their corresponding C++ values using `atoi()` and `atof()`. String literals and identifiers are duplicated using `strdup()` before being passed to the parser.

Identifiers follow the pattern:

```
letter (letter | digit)*
```

which permits names beginning with an alphabetic character or underscore followed by alphanumeric characters or underscores.

---

5. Syntax Analysis

Syntax analysis is implemented in `a5.y` using a context-free grammar.

The parser recognizes complete programs consisting of global declarations, function declarations, and function definitions. The grammar supports both functions with parameters and functions without parameters.

5.1 Variable Declarations

Variable declarations have the general structure:

```
type identifier ;
type identifier1, identifier2, ... ;
```

Supported types are:

- `int`

- `float`
- `bool`
- `string`
- `void`

Variables are inserted into the current symbol-table scope during parsing.

5.2 Expressions

The expression grammar supports:

- Addition
- Subtraction
- Multiplication
- Division
- Unary minus
- Relational expressions
- Logical AND
- Logical OR
- Logical NOT
- Parenthesized expressions
- Ternary expressions
- Variables
- Integer constants
- Floating-point constants
- String constants

The parser explicitly declares operator precedence and associativity. For example, multiplication and division have higher precedence than addition and subtraction, while logical operators have lower precedence. Unary minus and logical negation are given appropriate precedence levels.

The ternary operator is represented using:

```
condition ? true_expression : false_expression
```

and produces a `Ternary_Ast` node.

5.3 Statements

The language supports:

- Assignment
- `if`
- `if-else`
- `while`
- `do-while`
- Compound statements
- `read`
- `print`

- Function calls
- `return`

The grammar resolves the classic dangling-else ambiguity using the `IFX` precedence mechanism.

---

6. Symbol Table

The symbol table is implemented in `symbol_table.hh` and `symbol_table.cc`.

A `SymbolTableEntry` stores information such as:

- Symbol name
- Data type
- Symbol kind
- Source line number
- Function parameter information
- Definition status
- Stack offset
- Object size

The supported symbol categories are:

```
VARIABLE
FUNCTION
PARAMETER
```

6.1 Scope Management

The symbol table represents nested scopes using a vector of hash maps:

```
vector<unordered_map<string, SymbolTableEntry>> scopes
```

A new scope is created using `enter_scope()` and removed using `exit_scope()`.

Lookup searches from the innermost scope outward. This allows local variables to correctly shadow symbols in outer scopes.

6.2 Global Variables

Variables declared in the outermost scope are recorded separately in `global_vars`. This information is later used during assembly generation to distinguish global variables from stack-resident local variables.

6.3 Stack Offsets

The symbol table also assigns memory offsets to parameters and local variables.

Parameters use positive offsets relative to the frame pointer, while local variables use negative offsets.

For example, the implementation initializes the parameter offset counter to `8` and allocates local variables using decreasing negative offsets. Floating-point variables occupy 8 bytes while other variables use 4 bytes.

The symbol table preserves the completed scope of each function, together with its required stack allocation. This information is later used by the assembly generator to construct function stack frames.

---

7. Semantic Analysis

Semantic analysis is implemented through two complementary mechanisms:

1. Semantic checks performed while parsing.
2. AST-level type and structural checking after parsing.

The parser performs checks such as:

- Redeclaration of variables
- Redeclaration of functions
- Use of undeclared variables
- Use of undeclared functions
- Function/non-function name conflicts
- Function return-type mismatches
- Function parameter-count mismatches

For example, when a variable is used in an expression, the parser performs a symbol-table lookup and reports an undeclared-variable error if the symbol does not exist.

7.1 AST Type Checking

The AST performs additional semantic validation.

Examples include:

- Conditions of `if` statements must be boolean.
- Conditions of `while` statements must be boolean.
- Conditions of `do-while` statements must be boolean.
- Ternary conditions must be boolean.
- Both branches of a ternary expression must have matching types.
- Boolean values cannot be printed.
- Boolean and string values cannot be read.
- Function arguments must have the correct number and type.
- Function return statements must conform to the function's return type.

For example, `If_Else_Ast::check_ast()` explicitly verifies that the condition has type `bool`.

The ternary AST similarly verifies both the condition type and the compatibility of its two result expressions.

Function calls perform both argument-count and argument-type checking using the function information stored in the symbol table.

---

8. Abstract Syntax Tree

The Abstract Syntax Tree is implemented through the class hierarchy in `ast.hh` and `ast.cc`.

The base class is:

```
Ast
```

It defines common functionality such as:

- Data-type retrieval
- Semantic checking
- AST printing
- TAC generation

Each concrete syntactic construct is represented by a derived class.

8.1 Expression Nodes

The AST contains nodes for:

- `Name_Ast`
- `Int_Ast`
- `Float_Ast`
- `String_Ast`
- `Expression_Ast`
- `Relational_Ast`
- `Logical_Ast`
- `Logical_Not_Ast`
- `Uminus_Ast`
- `Ternary_Ast`

The binary arithmetic operators are represented using the `BinOp` enumeration:

```
OP_ADD
OP_SUB
```

```
OP_MULT
OP_DIV
```

Relational operators are represented using `RelOp`, and logical operations use `LogOp`.

8.2 Statement Nodes

Statement-level AST classes include:

- `Assignment_Ast`
- `Sequence_Ast`
- `Compound_Statement_Ast`
- `If_Else_Ast`
- `While_Ast`
- `Do_While_Ast`
- `Print_Ast`
- `Read_Ast`
- `Function_Call_Ast`
- `Return_Ast`

This object-oriented design makes the compiler extensible because each language construct encapsulates its own semantic validation, printing, and TAC-generation logic.

8.3 Function and Program Representation

Functions are represented by `Function_Ast`, which stores:

- Function name
- Return type
- Formal parameters
- Function body
- Declaration information

A program is represented using the program-level AST structure and contains the collection of functions.

The AST can also be printed into a `.ast` file, which provides a useful intermediate debugging and verification representation. The compiler driver enables this using the `--show-ast` option.

---

9. Three-Address Code

The next major stage is generation of Three-Address Code (TAC).

TAC simplifies the AST into a sequence of low-level operations in which complex expressions are decomposed into individual operations involving temporary variables.

The TAC framework is defined by `TAC_Statement` and several derived classes:

- `TAC_Assignment_Statement`
- `TAC_Goto_Statement`
- `TAC_Label_Statement`
- `TAC_IO_Statement`
- `TAC_Func_Call_Statement`
- `TAC_Return_Statement`
- `TAC_Func_Begin_Statement`
- `TAC_Func_End_Statement`

9.1 Temporary Variables

`TAC_Code` maintains counters for:

- Temporary variables
- Labels
- Special temporaries

and provides functions such as:

```
get_new_temp()
get_new_stemp()
get_new_label()
```

This ensures that intermediate values and control-flow labels have unique names.

For example, an expression such as:

```
a = b + c * d
```

can conceptually be transformed into:

```
t0 = c * d
t1 = b + t0
a = t1
```

This representation is significantly easier to translate into machine-level instructions.

---

10. TAC Generation for Control Flow

Control-flow constructs are converted into labels and conditional/unconditional jumps.

10.1 If-Else

An `if-else` statement is converted into:

1. Code for the condition.
2. A boolean inversion or conditional value.
3. A conditional jump.
4. The `then` block.
5. A jump over the `else` block.
6. The `else` block.
7. The final label.

The implementation explicitly creates fresh labels and temporary values for this process.

10.2 While Loop

A `while` loop generates:

```
L0:
    evaluate condition
    if false goto L1
    body
    goto L0
L1:
```

The implementation generates two labels and places the condition evaluation at the beginning of the loop.

10.3 Do-While Loop

The `do-while` loop differs because the body must execute at least once:

```
L0:
    body
    evaluate condition
    if true goto L0
```

This structure is directly reflected in the TAC generation implementation.

---

11. Ternary Expression Translation

The ternary operator is translated using labels and a temporary result.

For an expression:

```
c ? x : y
```

the compiler effectively generates control flow of the form:

```
evaluate c
if false goto L1

result = x
goto L2

L1:
result = y

L2:
```

The implementation uses separate TAC contexts for the condition, true expression, and false expression, while maintaining consistent temporary and label counters.

This demonstrates that the compiler treats the ternary operator as a control-flow construct rather than merely as a simple binary expression.

---

12. Function Calls and Returns

Functions are represented explicitly in TAC.

A function begins with a `TAC_Func_Begin_Statement` and ends with a `TAC_Func_End_Statement`. Function calls are represented by `TAC_Func_Call_Statement`.

For non-void functions, a temporary is allocated to hold the return value.

The compiler also handles argument passing. Arguments are evaluated and pushed onto the stack before the function is called. The arguments are processed in reverse order, after which the function is invoked.

Return values are mapped to target registers:

- Integer-like return values use `$v1`.
- Floating-point return values use `$f0`.
- Void functions do not require a return register.

The implementation then jumps to a function-specific epilogue.

---

13. Register Transfer Language

The RTL layer provides another abstraction between TAC and target assembly.

The RTL representation includes operations such as:

- Compute
- Load
- Immediate load
- Store
- Move
- Conditional move
- Goto
- Conditional goto
- Label
- Read
- Write
- Push
- Pop
- Function call
- Return
- Function begin
- Function end

The purpose of this intermediate representation is to make target-specific code generation easier while still keeping the representation more structured than raw assembly.

For example, an RTL operation may be represented conceptually as:

```
add: t0 <- t1, t2
```

or:

```
load: t0 <- x
```

The RTL generator then converts these abstract operations into MIPS instructions.

---

14. Register Allocation

The project implements a simple register-management system in `RTL_Code`.

Integer registers include:

```
$v0
$t0 ... $t8
```

Floating-point registers include:

```
$f2, $f4, ... $f30
```

The compiler maintains availability information for these registers and maps TAC temporaries to registers using `temp_map`.

The allocation process is simple:

1. Search for an existing register mapping for a temporary.
2. If none exists, allocate an available register.
3. Record the mapping.
4. Release the register once the value is no longer required.

If no suitable register is available, the implementation reports an "Out of integer registers" or "Out of float registers" error.

Although this is not a sophisticated graph-coloring register allocator, it provides a functional bridge between temporaries and physical target registers.

---

15. Integer and Floating-Point Operations

The compiler distinguishes integer and floating-point operations during RTL generation.

For integer arithmetic, standard MIPS instructions such as:

```
add
sub
mul
div
```

are generated.

Floating-point arithmetic uses double-precision operations such as:

```
add.d
sub.d
mul.d
div.d
```

The TAC generator determines whether an operation involves floating-point operands and chooses the appropriate RTL operation.

Relational operations involving floating-point values require special MIPS floating-point comparison instructions. For example:

```
slt.d
sle.d
seq.d
```

are mapped to appropriate MIPS floating-point comparison instructions such as:

```
c.lt.d
c.le.d
c.eq.d
```

---

16. Input and Output

The compiler supports `read` and `print` statements.

Different data types are mapped to appropriate SPIM/MIPS system-call conventions.

For example, integer output loads the appropriate syscall number into `$v0` and places the value in `$a0`. Floating-point output uses `$f12`, while string output uses `$a0` to hold the address of the string.

The compiler also maintains a mapping for string constants:

```
_str_0
_str_1
...
```

so that repeated string constants can be assigned consistent labels.

Input similarly distinguishes integer and floating-point operations and stores the result into the appropriate variable location.

---

17. Stack and Function Frame Management

One of the more substantial backend components is function stack management.

At the beginning of a function, the compiler:

1. Saves the return address.
2. Saves the previous frame pointer.
3. Establishes a new frame pointer.
4. Allocates space for local variables.

The generated structure is conceptually:

```
sw   $ra, 0($sp)
sw   $fp, -4($sp)
sub  $fp, $sp, 4
sub  $sp, $sp, allocation
```

The required stack allocation is obtained from the symbol table.

At function exit, an epilogue label is generated and the stack frame is restored:

```
epilogue_function:
    add $sp, $sp, allocation
    lw  $fp, -4($sp)
    lw  $ra, 0($sp)
    jr  $ra
```

This design allows all return statements to jump to a common epilogue rather than duplicating stack-restoration code.

---

18. Assembly Representation

The final assembly abstraction is implemented through `Asm_Statement`.

The main classes are:

- `Directive_Asm_Statement`
- `Label_Asm_Statement`
- `Instruction_Asm_Statement`
- `Memory_Asm_Statement`
- `Syscall_Asm_Statement`

This provides an object-oriented representation of assembly instructions instead of directly concatenating strings throughout the compiler.

For example, a standard instruction is represented by:

```
opcode
operand1
```

```
operand2
operand3
```

while memory instructions additionally store:

- Register
- Offset
- Base register

The assembly classes eventually serialize these objects into valid SPIM/MIPS assembly syntax.

---

19. Compiler Driver

The complete compilation process is coordinated by `main.cc`.

The compiler supports several command-line options:

```
--show-tokens
--show-ast
--sa-scan
--sa-parse
```

The `--sa-scan` option allows the lexical analyzer to be executed independently, while `--sa-parse` allows syntax parsing without semantic checking.

The normal compilation path is:

```
yyparse()
      |
      v
AST construction
      |
      v
AST semantic validation
      |
      v
TAC generation
      |
      v
RTL generation
      |
      v
Assembly generation
```

The compiler can generate:

- `.toks` — token output
- `.ast` — AST output
- `.rtl` — RTL output
- `.spim` — final assembly output

The driver invokes the corresponding generation functions in sequence.

---

20. Example Compilation Flow

Consider a simple source fragment:

```
int main() {
    int a;
    int b;

    a = 5;
    b = a + 10;

    print b;
}
```

The compilation proceeds through several representations.

Source Level

```
a = 5;
b = a + 10;
print b;
```

AST

The statements are represented approximately as:

```
Assignment
    |
    +-- Name(a)
    +-- Integer(5)

Assignment
    |
    +-- Name(b)
    +-- Expression(+)
          |
          +-- Name(a)
```

```
          +-- Integer(10)

Print
    |
    +-- Name(b)
```

TAC

The expression is decomposed into temporary operations:

```
a = 5
t0 = a + 10
b = t0
write b
```

RTL

The operations are converted into loads, computations, stores, and I/O operations:

```
load constant
store a
load a
load constant
add
store b
load b
write
```

MIPS

Finally, the RTL is translated into MIPS/SPIM instructions involving registers, memory accesses, and system calls.

This multi-stage representation makes the compiler easier to debug because each stage can be inspected independently.

---

21. Testing and Debugging Support

The compiler provides explicit intermediate outputs that are useful during development.

The token output can be enabled using:

```
--show-tokens
```

The AST output can be enabled using:

```
--show-ast
```

The standalone scanning and parsing options allow the front-end components to be tested independently.

The repository also contains a `testcases/valid` directory, providing programs intended for validating the compiler's behavior.

This staged testing methodology is particularly useful for compiler development because an incorrect final assembly output can be traced back through:

```
Assembly
   ↑
RTL
   ↑
TAC
   ↑
AST
   ↑
Parser
   ↑
Lexer
```

---

22. Design Decisions

Several important design decisions are visible in the implementation.

22.1 Object-Oriented Intermediate Representations

AST, TAC, RTL, and assembly instructions are represented using class hierarchies.

This provides polymorphic interfaces such as:

```
check_ast()
gen_tac_code()
gen_rtl_code()
gen_asm_code()
```

and makes it possible to add new language constructs without redesigning the complete compiler.

22.2 Multiple Intermediate Representations

Instead of directly translating the AST to assembly, the compiler uses both TAC and RTL.

This separates:

- language-level semantics,
- machine-independent intermediate operations,
- machine/register-level operations,
- final assembly syntax.

This is a conventional compiler architecture and makes the backend considerably easier to reason about.

22.3 Symbol Table Reuse Across Backend Stages

The symbol table is not discarded after semantic analysis.

It is preserved and used later to determine:

- Global variables
- Local variable locations
- Parameter locations
- Variable sizes
- Function stack allocation

This allows the backend to translate a symbolic variable name into the correct stack-frame location during assembly generation.

22.4 Common Function Epilogues

Return statements jump to a common function-specific epilogue. This avoids duplicating frame restoration instructions for every return statement and provides a clean implementation of function returns.

---

23. Challenges in the Implementation

Implementing the complete compiler pipeline involves several non-trivial problems.

23.1 Maintaining Type Information

Type information must be available across multiple stages. The AST therefore stores the data type of expressions and uses it for semantic validation and later TAC generation.

23.2 Scope Management

Functions introduce new scopes, and symbols must be looked up from the innermost scope outward. The symbol table handles this using a stack of hash maps.

23.3 Translating Structured Control Flow

High-level constructs such as `if`, `while`, `do-while`, and ternary expressions must be transformed into labels and jumps. This requires careful management of temporary variables and labels.

23.4 Function Calling Convention

Function calls require coordination between:

- argument evaluation,
- stack-based argument passing,
- return registers,
- function stack frames,
- saved `$ra` and `$fp`,
- restoration of the stack.

The implementation explicitly handles these operations in the TAC and RTL layers.

23.5 Floating-Point Support

Floating-point values require different register files and MIPS instructions from integer values. The backend therefore maintains separate integer and floating-point register pools and generates appropriate `.d` operations.

---

24. Limitations and Possible Improvements

Although the compiler implements a broad set of features, several areas could be improved.

24.1 Register Allocation

The current register allocator uses a straightforward availability-based allocation scheme. A more sophisticated implementation could use:

- liveness analysis,
- interference graphs,
- graph coloring,
- spilling to memory.

This would make the compiler more robust for programs containing many simultaneously live temporaries.

24.2 Optimization

The current pipeline primarily performs translation rather than aggressive optimization.

Potential optimizations include:

- Constant folding

- Constant propagation
- Dead-code elimination
- Common subexpression elimination
- Copy propagation
- Peephole optimization
- Control-flow simplification

These could be implemented between TAC and RTL.

24.3 Error Recovery

The parser reports syntax errors using `yyerror()`, but the current implementation does not implement extensive error recovery. A production compiler could recover from errors and continue parsing to report multiple errors in one compilation.

24.4 Memory Management

Several AST and intermediate-representation objects are dynamically allocated. A more modern implementation could use smart pointers to simplify ownership and reduce the possibility of memory-management errors.

24.5 Backend Abstraction

The current backend is specifically designed around MIPS/SPIM conventions. A more portable compiler could introduce a target-independent backend interface and implement separate target modules for MIPS, RISC-V, x86-64, or ARM.

---

25. Learning Outcomes

This project provides practical experience with nearly every major phase of a compiler.

The implementation demonstrates understanding of:

- Regular expressions and lexical analysis
- Context-free grammars
- Parser construction
- Operator precedence
- Abstract syntax trees
- Symbol tables
- Scope management
- Static semantic analysis
- Type checking
- Intermediate representation
- Three-address code
- Control-flow translation
- Temporary and label management
- Register allocation
- Stack-frame construction

- Calling conventions
- MIPS assembly
- Compiler-driver design

The most important aspect of the project is the connection between these individual concepts. Rather than implementing isolated compiler components, the project connects them into a complete source-to-target compilation pipeline.

---

26. Conclusion

The CS306 Assignment 5 project implements a complete compiler pipeline for a small procedural programming language. Starting from a source program, the compiler performs lexical analysis using a Flex-style lexer and syntax analysis using a Yacc/Bison grammar. The parser constructs an AST while interacting with a symbol table to perform early semantic checks.

The AST subsequently performs additional semantic and type validation before generating Three-Address Code. TAC provides a simpler representation of expressions, control flow, function calls, and I/O. The compiler then translates TAC into a Register Transfer Language that explicitly represents register operations, memory accesses, control flow, function calls, and stack operations.

Finally, RTL instructions are converted into MIPS/SPIM assembly. The backend includes stack-frame construction, local-variable offsets, parameter handling, integer and floating-point registers, function calls, return values, and system-call-based input/output.

The project therefore demonstrates the complete progression:

```
High-Level Language
        ↓
Lexical Tokens
        ↓
Parse Tree / AST
        ↓
Semantic Representation
        ↓
Three-Address Code
        ↓
Register Transfer Language
        ↓
MIPS / SPIM Assembly
```

A particularly valuable aspect of the implementation is the use of multiple intermediate representations. This separates the concerns of language parsing, semantic analysis, machine-independent code generation, register management, and target-specific assembly generation. The resulting architecture provides a strong foundation for future extensions such as optimization passes, more advanced register allocation, additional target architectures, and improved error recovery.

Overall, the project provides a practical implementation of the fundamental concepts of compiler construction and demonstrates how a high-level programming language can be systematically transformed into executable low-level code.

---

27. Project Structure

The principal source files and their responsibilities are summarized below:

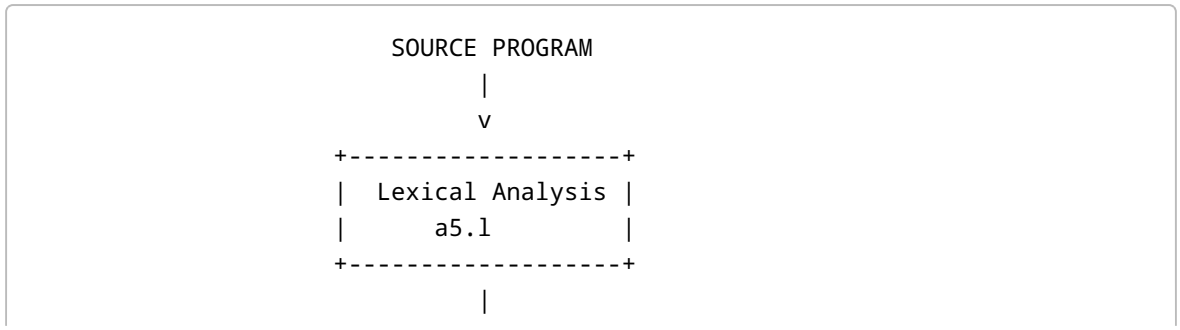
File	Responsibility
`a5.l`	Lexical analysis
`a5.y`	Grammar, parsing, and parser-level semantic checks
`symbol_table.hh`	Symbol-table declarations
`symbol_table.cc`	Scope, symbol lookup, offsets, and stack allocation
`ast.hh`	AST class hierarchy
`ast.cc`	AST validation, printing, and TAC generation
`tac.hh`	TAC representation
`tac.cc`	TAC generation and RTL translation
`rtl.hh`	RTL representation
`rtl.cc`	RTL generation and assembly translation
`asm.hh`	Assembly representation
`asm.cc`	Assembly instruction serialization
`main.cc`	Compiler driver and pipeline orchestration
`testcases/valid`	Valid source programs for testing

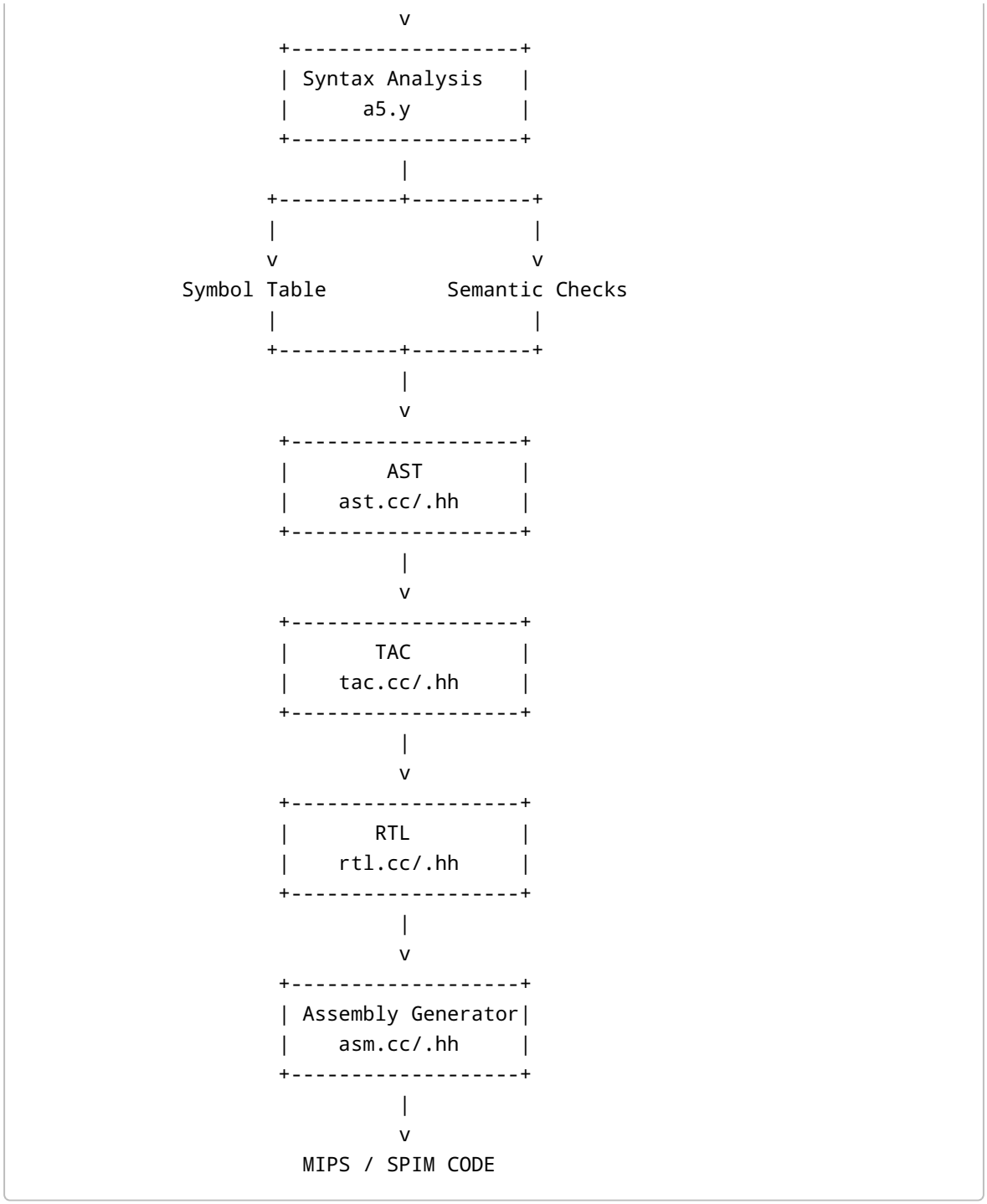
The complete implementation and these components are present in the Assignment 5 repository.

---

28. Final Compilation Pipeline

The final architecture of the implemented compiler can be summarized as:





This architecture captures the complete functionality implemented in the project and illustrates how the different compiler phases cooperate to transform a high-level source program into executable target code.